

Beer Production Australia

In this dataset, we have historical monthly dataset of beer production in Australia from the year 1956 to 1995 (three quarters). Our job is to predict the beer production in 1995 (final quarter) and the entire years of 1996 and 1997. We will be using time series analysis for this purpose.

```
In [1]: #First we import the dataset

import pandas as pd

data = pd.read_csv('C:\\Users\\sujoydutta\\Desktop\\DA material\\Datasets for ML\\Time S
data.head()
```

```
Out[1]:
```

	Month	Monthly beer production
0	1956-01	93.2
1	1956-02	96.0
2	1956-03	95.2
3	1956-04	77.1
4	1956-05	70.9

```
In [2]: # Converting to date time format
data['Month'] = pd.to_datetime(data['Month'])
data.set_index('Month', inplace=True)
data.head()
```

```
Out[2]:
```

	Monthly beer production
Month	
1956-01-01	93.2
1956-02-01	96.0
1956-03-01	95.2
1956-04-01	77.1
1956-05-01	70.9

```
In [3]: # Let us separate the prediction dataset
start_predict = '1995-10-01'
end_predict = '1997-12-01'

predict_df = data.loc[start_predict:end_predict]

predict_df.head()
```

```
Out[3]:
```

	Monthly beer production
Month	
1995-10-01	NaN
1995-11-01	NaN

1995-12-01	NaN
1996-01-01	NaN
1996-02-01	NaN

```
In [4]: # Now let us remove the empty values from the main dataset
main = data.dropna()
main.head()
```

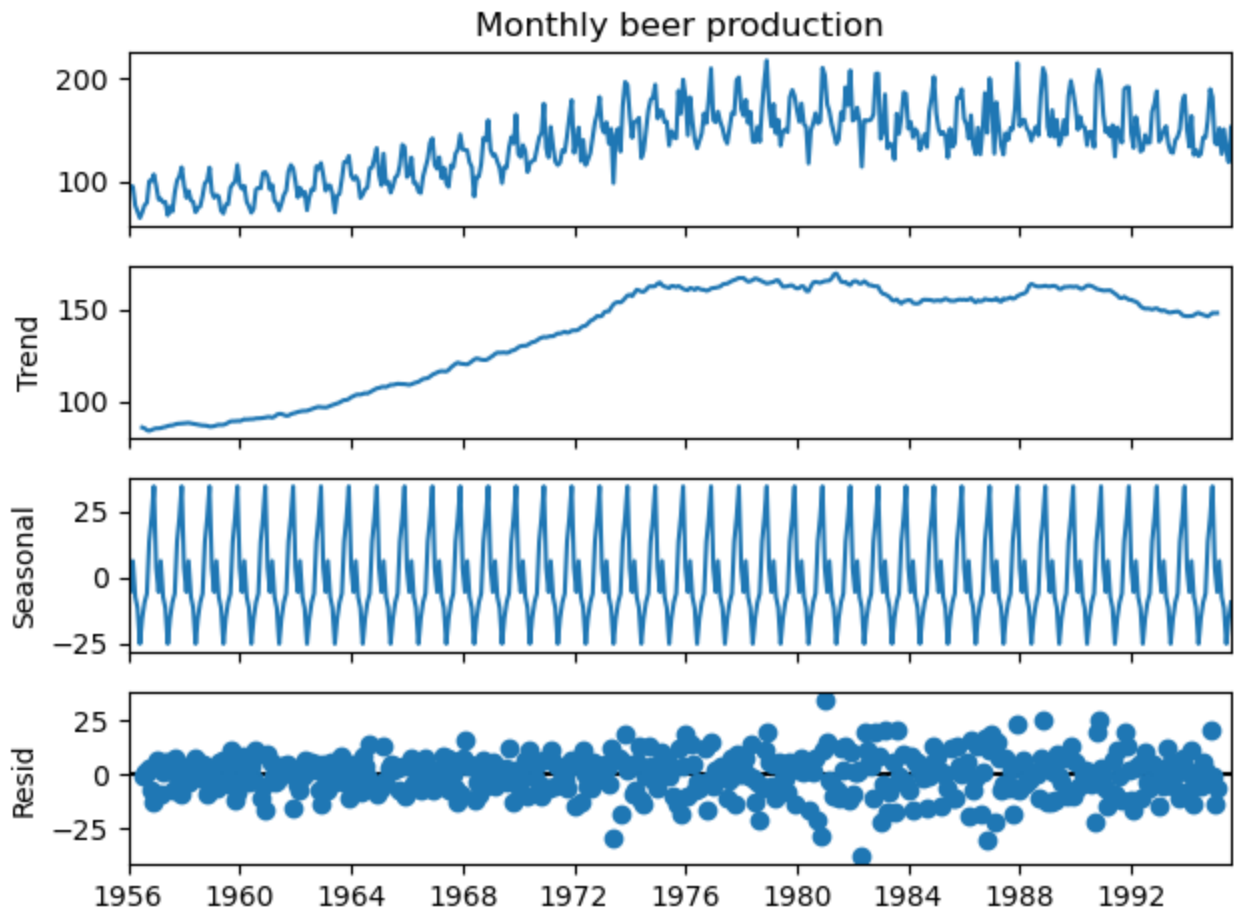
Out[4]: **Monthly beer production**

Month	
1956-01-01	93.2
1956-02-01	96.0
1956-03-01	95.2
1956-04-01	77.1
1956-05-01	70.9

```
In [5]: # Now we decompose the time series
from statsmodels.tsa.seasonal import seasonal_decompose

result = seasonal_decompose(main['Monthly beer production'], model='additive', period=12)
```

```
In [6]: # And we plot the decomposed components
import matplotlib.pyplot as plt
result.plot()
plt.show()
```



```
In [7]: # We do a Ljung-Box test to see if seasonality is there or not

from statsmodels.stats.diagnostic import acorr_ljungbox

ljung_box_result = acorr_ljungbox(main['Monthly beer production'].dropna(), lags=[12], r
print(ljung_box_result)

          lb_stat  lb_pvalue
12  2751.330066      0.0
```

Observation: Seeing the p-value is very much less than 0.05, so we can conclude that there is strong seasonality in the data. A strong Autocorrelation has been observed that suggests some form of seasonality is there.

```
In [8]: # Let us partition the dataset into training and testing sets
train_df = main.loc['1956-01-01':'1994-09-01']
test_df = main.loc['1994-10-01':]
```

```
In [9]: # Now we set SARIMA parameters
sarima_order = (1, 1, 1)          # (p, d, q) for non-seasonal part
sarima_seasonal_order = (1, 1, 1, 12) # (P, D, Q, m) for seasonal part as monthly data
```

```
In [10]: # Now we train SARIMA model
from statsmodels.tsa.statespace.sarimax import SARIMAX
sarima_model = SARIMAX(train_df['Monthly beer production'], order=sarima_order, seasonal
sarima_fit = sarima_model.fit(dispatch=False)
```

```
C:\Users\sujoydutta\anaconda\Lib\site-packages\statsmodels\tsa\base\tsa_model.py:473: ValueWarning: No frequency information was provided, so inferred frequency MS will be used.
  self._init_dates(dates, freq)
C:\Users\sujoydutta\anaconda\Lib\site-packages\statsmodels\tsa\base\tsa_model.py:473: ValueWarning: No frequency information was provided, so inferred frequency MS will be used.
  self._init_dates(dates, freq)
```

```
In [11]: # Now we forecast with SARIMA model
sarima_predictions = sarima_fit.predict(start=test_df.index[0], end=test_df.index[-1])
sarima_predictions
```

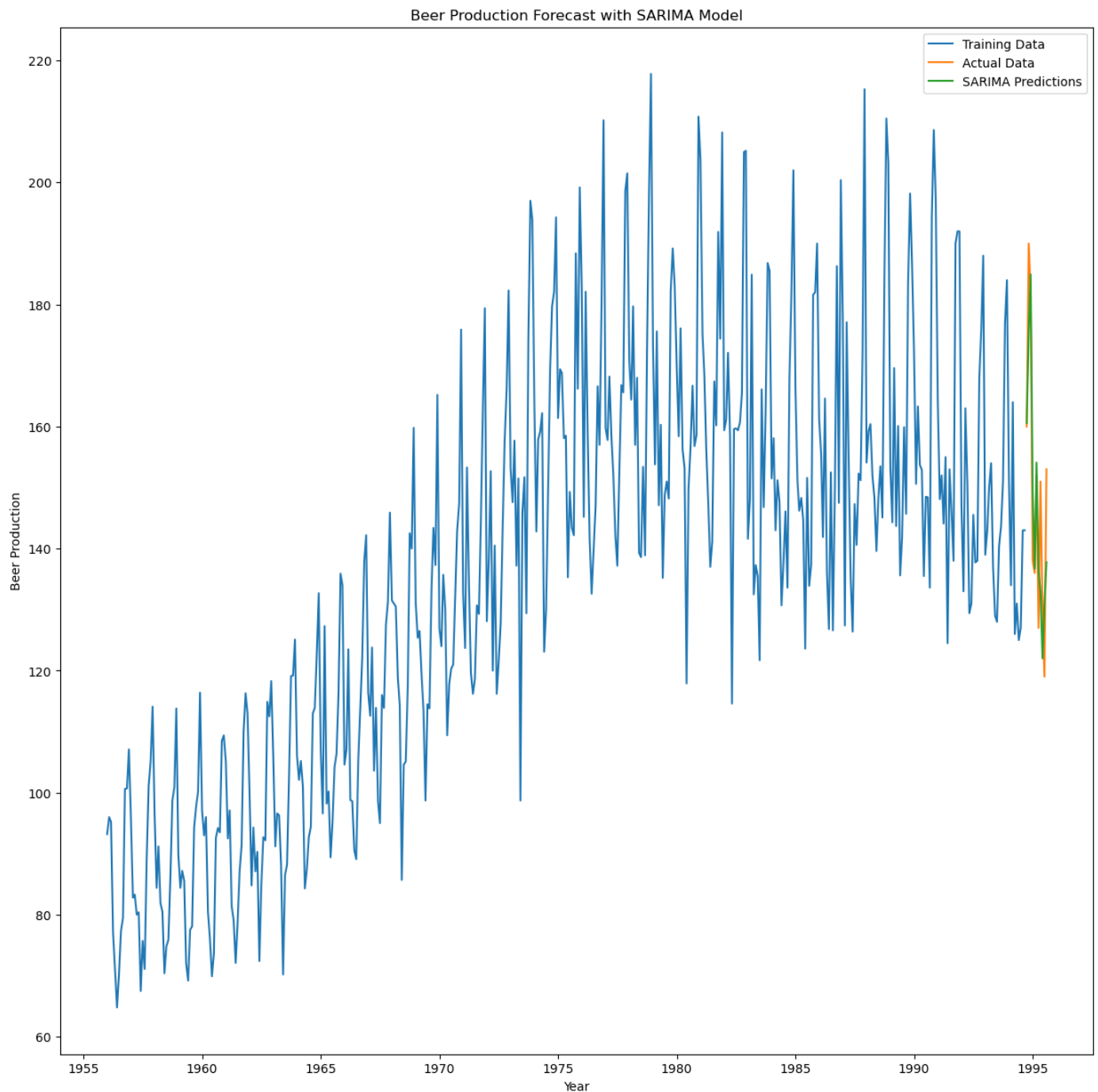
```
Out[11]: 1994-10-01    160.589719
1994-11-01    175.509216
1994-12-01    184.938281
1995-01-01    148.815859
1995-02-01    136.719958
1995-03-01    154.109322
1995-04-01    136.440371
1995-05-01    133.051356
1995-06-01    122.003718
1995-07-01    131.172931
1995-08-01    137.770342
Freq: MS, Name: predicted_mean, dtype: float64
```

```
In [12]: # Let us see the RMSE for SARIMA model
import numpy as np
from sklearn.metrics import mean_squared_error
rmse = np.sqrt(mean_squared_error(test_df['Monthly beer production'], sarima_predictions
print(f'SARIMA RMSE: {rmse}')

SARIMA RMSE: 10.428336163270851
```

```
In [29]: # Now let us plot actual vs. predicted for the SARIMA model
plt.figure(figsize=(15, 15))
plt.plot(train_df.index, train_df['Monthly beer production'], label='Training Data')
```

```
plt.plot(test_df.index, test_df['Monthly beer production'], label='Actual Data')
plt.plot(test_df.index, sarima_predictions, label='SARIMA Predictions')
plt.title('Beer Production Forecast with SARIMA Model')
plt.xlabel('Year')
plt.ylabel('Beer Production')
plt.legend()
plt.show()
```



```
In [13]: # Now we train an Exponential Smoothing model
from statsmodels.tsa.holtwinters import ExponentialSmoothing
ets_model = ExponentialSmoothing(train_df['Monthly beer production'], seasonal='add', se
ets_fit = ets_model.fit())
```

C:\Users\sujoydutta\anaconda\Lib\site-packages\statsmodels\tsa\base\tsa_model.py:473: ValueWarning: No frequency information was provided, so inferred frequency MS will be used.

```
self._init_dates(dates, freq)
```

```
In [14]: # Now we forecast with ETS model
ets_predictions = ets_fit.forecast(len(test_df))
ets_predictions
```

```
Out[14]: 1994-10-01    164.532889
1994-11-01    174.980778
1994-12-01    185.456916
1995-01-01    149.498033
1995-02-01    139.100828
1995-03-01    153.644358
1995-04-01    140.214865
1995-05-01    135.310623
1995-06-01    122.976059
1995-07-01    133.519347
1995-08-01    138.357023
Freq: MS, dtype: float64
```

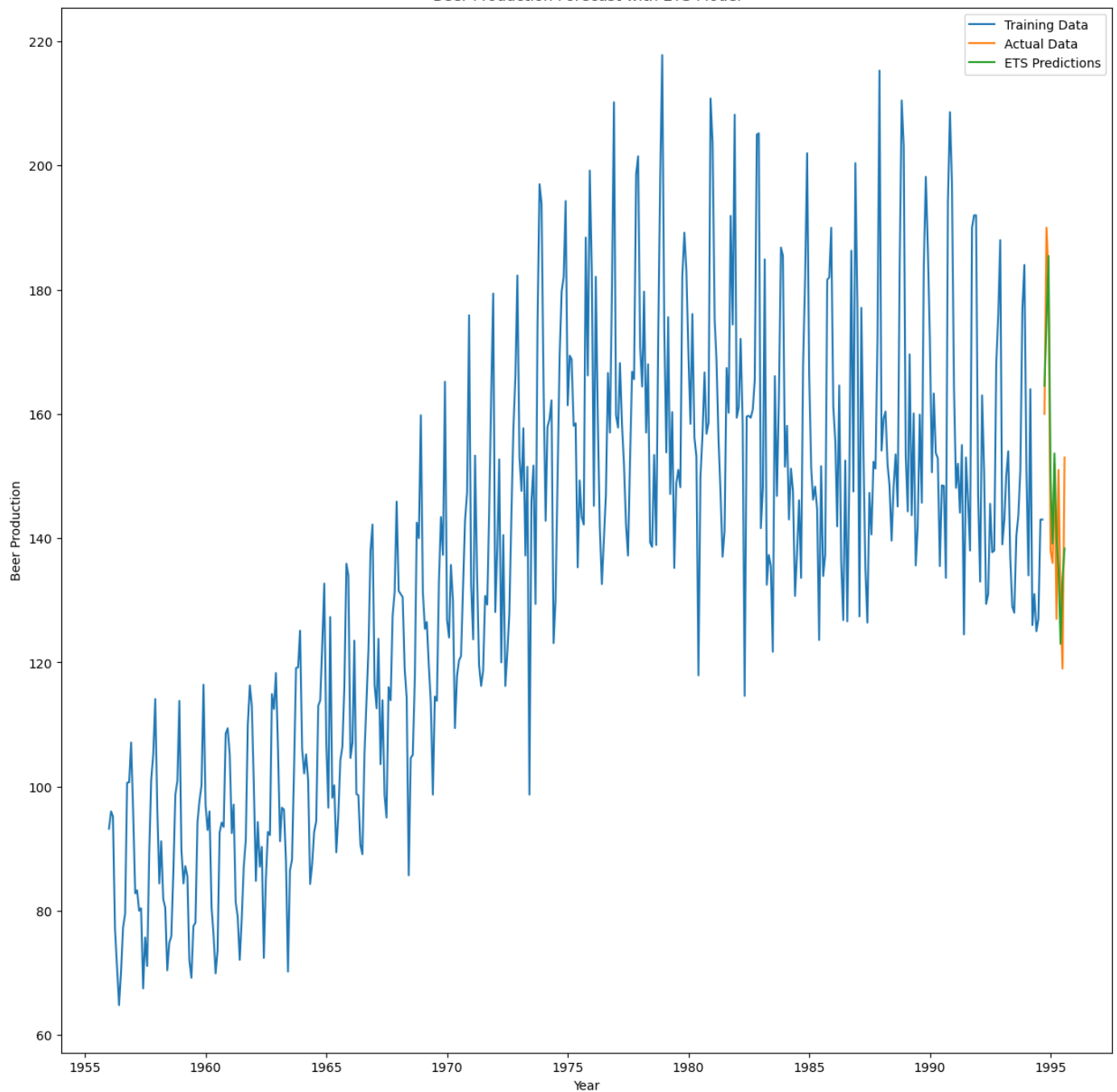
```
In [16]: # Let us see the RMSE for ETS model

rmse = np.sqrt(mean_squared_error(test_df['Monthly beer production'], ets_predictions))
print(f'ETS RMSE: {rmse}')

ETS RMSE: 10.862577061134415
```

```
In [28]: # Now let us plot actual vs. predicted for the ETS model
plt.figure(figsize=(15, 15))
plt.plot(train_df.index, train_df['Monthly beer production'], label='Training Data')
plt.plot(test_df.index, test_df['Monthly beer production'], label='Actual Data')
plt.plot(test_df.index, ets_predictions, label='ETS Predictions')
plt.title('Beer Production Forecast with ETS Model')
plt.xlabel('Year')
plt.ylabel('Beer Production')
plt.legend()
plt.show()
```

Beer Production Forecast with ETS Model



```
In [18]: #let us train an Auto ARIMA model
from pmdarima import auto_arima

auto_sarima_model = auto_arima(train_df['Monthly beer production'], seasonal=True, m=12,
```

Performing stepwise search to minimize aic

```
ARIMA(2,1,2) (1,0,1) [12] intercept : AIC=3475.331, Time=3.67 sec
ARIMA(0,1,0) (0,0,0) [12] intercept : AIC=4078.080, Time=0.13 sec
ARIMA(1,1,0) (1,0,0) [12] intercept : AIC=3815.045, Time=0.59 sec
ARIMA(0,1,1) (0,0,1) [12] intercept : AIC=3846.891, Time=0.80 sec
ARIMA(0,1,0) (0,0,0) [12] : AIC=4076.094, Time=0.10 sec
ARIMA(2,1,2) (0,0,1) [12] intercept : AIC=3806.155, Time=1.33 sec
ARIMA(2,1,2) (1,0,0) [12] intercept : AIC=inf, Time=3.09 sec
ARIMA(2,1,2) (2,0,1) [12] intercept : AIC=3475.765, Time=8.75 sec
ARIMA(2,1,2) (1,0,2) [12] intercept : AIC=3565.780, Time=8.23 sec
ARIMA(2,1,2) (0,0,0) [12] intercept : AIC=3959.945, Time=0.78 sec
ARIMA(2,1,2) (0,0,2) [12] intercept : AIC=3722.423, Time=3.10 sec
ARIMA(2,1,2) (2,0,0) [12] intercept : AIC=3563.193, Time=4.17 sec
ARIMA(2,1,2) (2,0,2) [12] intercept : AIC=inf, Time=9.08 sec
ARIMA(1,1,2) (1,0,1) [12] intercept : AIC=inf, Time=2.58 sec
ARIMA(2,1,1) (1,0,1) [12] intercept : AIC=3481.546, Time=3.21 sec
```

```

ARIMA(3,1,2) (1,0,1) [12] intercept      : AIC=3493.004, Time=4.00 sec
ARIMA(2,1,3) (1,0,1) [12] intercept      : AIC=3422.917, Time=3.48 sec
ARIMA(2,1,3) (0,0,1) [12] intercept      : AIC=3806.596, Time=1.73 sec
ARIMA(2,1,3) (1,0,0) [12] intercept      : AIC=inf, Time=2.74 sec
ARIMA(2,1,3) (2,0,1) [12] intercept      : AIC=3423.695, Time=8.29 sec
ARIMA(2,1,3) (1,0,2) [12] intercept      : AIC=3418.316, Time=8.43 sec
ARIMA(2,1,3) (0,0,2) [12] intercept      : AIC=3724.187, Time=4.49 sec
ARIMA(2,1,3) (2,0,2) [12] intercept      : AIC=inf, Time=7.95 sec
ARIMA(1,1,3) (1,0,2) [12] intercept      : AIC=3472.495, Time=6.23 sec
ARIMA(3,1,3) (1,0,2) [12] intercept      : AIC=3422.169, Time=7.64 sec
ARIMA(2,1,4) (1,0,2) [12] intercept      : AIC=3458.493, Time=9.65 sec
ARIMA(1,1,2) (1,0,2) [12] intercept      : AIC=inf, Time=6.14 sec
ARIMA(1,1,4) (1,0,2) [12] intercept      : AIC=3473.279, Time=7.36 sec
ARIMA(3,1,2) (1,0,2) [12] intercept      : AIC=3477.474, Time=8.22 sec
ARIMA(3,1,4) (1,0,2) [12] intercept      : AIC=3410.076, Time=10.14 sec
ARIMA(3,1,4) (0,0,2) [12] intercept      : AIC=3701.063, Time=10.16 sec
ARIMA(3,1,4) (1,0,1) [12] intercept      : AIC=3457.423, Time=4.54 sec
ARIMA(3,1,4) (2,0,2) [12] intercept      : AIC=inf, Time=12.74 sec
ARIMA(3,1,4) (0,0,1) [12] intercept      : AIC=3782.495, Time=4.22 sec
ARIMA(3,1,4) (2,0,1) [12] intercept      : AIC=3417.295, Time=9.00 sec
ARIMA(4,1,4) (1,0,2) [12] intercept      : AIC=3466.015, Time=10.73 sec
ARIMA(3,1,5) (1,0,2) [12] intercept      : AIC=inf, Time=12.97 sec
ARIMA(2,1,5) (1,0,2) [12] intercept      : AIC=3445.467, Time=9.60 sec
ARIMA(4,1,3) (1,0,2) [12] intercept      : AIC=3415.897, Time=9.63 sec
ARIMA(4,1,5) (1,0,2) [12] intercept      : AIC=inf, Time=14.21 sec
ARIMA(3,1,4) (1,0,2) [12] intercept      : AIC=3412.031, Time=9.61 sec

```

Best model: ARIMA(3,1,4) (1,0,2) [12] intercept
Total fit time: 253.872 seconds

```

In [19]: # Let us set the best ARIMA model parameters
best_arma_order = (3, 1, 4)
best_seasonal_order = (1, 0, 2, 12)

```

```

In [20]: # And we train the best ARIMA model
best_arma_model = SARIMAX(train_df['Monthly beer production'],
                           order=best_arma_order,
                           seasonal_order=best_seasonal_order,
                           trend='c')
best_arma_fit = best_arma_model.fit(dispatch=False)

```

```

C:\Users\sujoydutta\anaconda\Lib\site-packages\statsmodels\tsa\base\tsa_model.py:473: ValueWarning: No frequency information was provided, so inferred frequency MS will be used.
  self._init_dates(dates, freq)
C:\Users\sujoydutta\anaconda\Lib\site-packages\statsmodels\tsa\base\tsa_model.py:473: ValueWarning: No frequency information was provided, so inferred frequency MS will be used.
  self._init_dates(dates, freq)
C:\Users\sujoydutta\anaconda\Lib\site-packages\statsmodels\tsa\statespace\sarimax.py:978: UserWarning: Non-invertible starting MA parameters found. Using zeros as starting parameters.
  warn('Non-invertible starting MA parameters found.')
C:\Users\sujoydutta\anaconda\Lib\site-packages\statsmodels\base\model.py:607: ConvergenceWarning: Maximum Likelihood optimization failed to converge. Check mle_retvals
  warnings.warn("Maximum Likelihood optimization failed to ")

```

```

In [21]: # Now we forecast with the best ARIMA model
best_arma_predictions = best_arma_fit.predict(start=test_df.index[0], end=test_df.index[-1])
best_arma_predictions

```

```

Out[21]: 1994-10-01    155.894392
         1994-11-01    180.502951
         1994-12-01    181.038334
         1995-01-01    140.063418

```

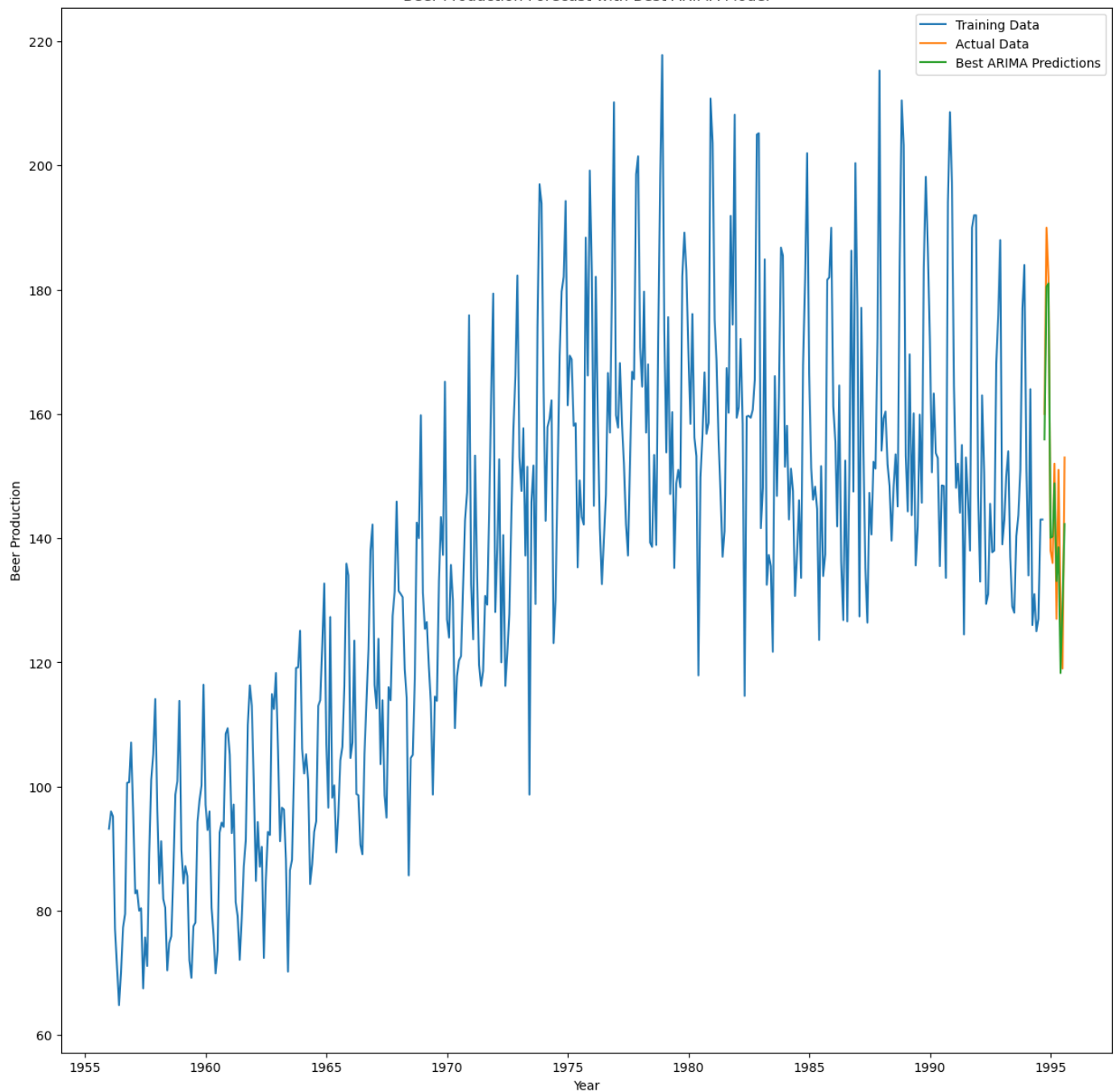
```
1995-02-01    140.286571
1995-03-01    148.835465
1995-04-01    133.087319
1995-05-01    138.539069
1995-06-01    118.270473
1995-07-01    129.035631
1995-08-01    142.280233
Freq: MS, Name: predicted_mean, dtype: float64
```

```
In [23]: # And we obtain RMSE for the best ARIMA model
best_arma_rmse = np.sqrt(mean_squared_error(test_df['Monthly beer production'], best_ar
print(f'Best ARIMA RMSE: {best_arma_rmse}')
```

```
Best ARIMA RMSE: 7.8979903197971195
```

```
In [26]: # Now let us plot actual vs. predicted for the best ARIMA model
plt.figure(figsize=(15, 15))
plt.plot(train_df.index, train_df['Monthly beer production'], label='Training Data')
plt.plot(test_df.index, test_df['Monthly beer production'], label='Actual Data')
plt.plot(test_df.index, best_arma_predictions, label='Best ARIMA Predictions')
plt.title('Beer Production Forecast with Best ARIMA Model')
plt.xlabel('Year')
plt.ylabel('Beer Production')
plt.legend()
plt.show()
```

Beer Production Forecast with Best ARIMA Model



```
In [30]: # Now we finally predict for the periods with missing values
predict_start_date = predict_df.index[0]
predict_end_date = predict_df.index[-1]
predicted_values = best_arma_fit.predict(start=predict_start_date, end=predict_end_date)
predicted_values
```

```
Out[30]: 1995-10-01    159.213862
1995-11-01    180.156181
1995-12-01    174.264691
1996-01-01    143.856084
1996-02-01    138.382281
1996-03-01    142.916674
1996-04-01    137.342296
1996-05-01    134.980006
1996-06-01    113.770702
1996-07-01    133.264917
1996-08-01    137.234439
1996-09-01    126.286351
1996-10-01    162.790320
1996-11-01    173.695790
1996-12-01    172.076562
```

1997-01-01	146.842676
1997-02-01	131.598392
1997-03-01	142.494972
1997-04-01	139.351221
1997-05-01	127.920647
1997-06-01	114.977277
1997-07-01	134.041972
1997-08-01	130.256223

Freq: MS, Name: predicted_mean, dtype: float64